R Notebook

Data manipulation

Introduction:

Data needs to be examined and any problems fixed before analysis can be done.In statistics, we focus on four areas.

1. Data accuracy
    - make sure data types are correct
    - check for typos
    - check for nonsensical data
    - check categories make sense
    - correct problems if possible or delete
    - reverse code items if needed
    - score instruments if needed
    - keep track of what you do so you can be transparent
2. Missing data
3. outliers
4. Statistical assumptions

This assignment shows some more steps to take to get data ready for analysis.

Directions:

Complete each step of the assignment below.

Package management in R

```
# keep a list of the packages used in this script
packages <- c("tidyverse","rio","jmv")
```

This next code block has eval=FALSE because you don't want to run it when knitting the file. Installing packages when knitting an R notebook can be problematic.

```
# check each of the packages in the list and install them if they're not installed already
for (i in packages){
  if(! i %in% installed.packages()){
    install.packages(i,dependencies = TRUE)
  }
  # show each package that is checked
  print(i)
}
```

```r
# load each package into memory so it can be used in the script
for (i in packages){
  library(i,character.only=TRUE)
  # show each package that is loaded
  print(i)
}
```

```
## -- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
## v dplyr     1.1.4     v readr     2.1.6
## v forcats   1.0.1     v stringr   1.6.0
## v ggplot2   4.0.1     v tibble    3.3.0
## v lubridate 1.9.4     v tidyr     1.3.2
## v purrr     1.2.0
## -- Conflicts ------------------------------------------ tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
## [1] "tidyverse"
## [1] "rio"
## [1] "jmv"
```

A note about working with packages and functions in R: When installing packages you may see notifications that SomePackage::function() masks OtherPackage::function(). What that means is that two different packages used the same function name. The functions may or may not do the same thing, but for whatever reason the pogrammers of each package happened to choose the same name for a function in their respective package. The last package installed will be the version of that function that will be used if only the function name is used in your R code. The SomePackage::function() notation with the colons in it means to use the function named function() from the package named SomePackage. I think I have decided when I write R code that I will always try to stick to that notation. That way it's very clear which package a desired function comes from and there will be no problems with masking if different packages used the same function name.

Open data file

The rio package works for importing several different types of data files. We're going to use it in this class. There are other packages which can be used to open datasets in R. You can see several options by clicking on the Import Dataset menu under the Environment tab in RStudio. (For a csv file like we have this week we'd use either From Text(base) or From Text (readr). Try it out to see the menu dialog.)

```r
# import the Week3.rds dataset into RStudio

# Using the file.choose() command allows you to select a file to import from another folder.
# dataset <- rio::import(file.choose())

# This command will allow us to import the rds file included in our project folder.
dataset <- rio::import("Week3.rds")
```

```
## Warning: Missing ‘trust‘ will be set to FALSE by default for RDS in 2.0.0.
```

Examine the dataset

Examine the dataset and answer the following questions. Enter any code you use in the following code block.

- How many variables are in the dataset?
  - Answer: 11
- List each variable and the level of measurement. Note if there are any issues you see with the data in that variable. (The first one is done for you.)
  - sex - nominal, no issues noted
- height – ratio, no issues noted
- weight – ratio, no issues noted
- lvst1 – ordinal, no issues noted
- lvst2 – ordinal, no issues noted
- lvst3 – ordinal, no issues noted
- lvst4 – ordinal, no issues noted
- frst1 – ordinal, no issues noted
- frst2 – ordinal, no issues noted
- frst3 – ordinal, no issues noted

```
ncol(dataset)
```

```
## [1] 11
```

Calculate a new column

The dataset includes variables for height and weight. Calculate the BMI for each participant by executing the following code block. The formula for BMI is weight (kg)/height (m)^2. Round the result to one decimal place.

- What level of measurement is the new bmi variable?
  - Answer: ratio

```
dataset <- dataset %>% mutate(bmi = round(weight / ((height / 100)^2), 1))
```

Reverse code a column

The dataset contains questions for two fictitious instruments (questionnaires): 1) the Love of Statistics Scale, and 2) the Fear of Statistics Scale.

The Love of Statistics Scale consists of 4 questions rated on a scale of 1-5. Before the instrument can be scored, the last question needs to be reverse coded. Execute the following code block to reverse code the last question in the Love of Statistics Scale.

- What level of measurement is the new lvst4reverse variable?
  - Answer: ordinal

```
dataset <- dataset %>%
  mutate(lvst4reverse = recode(lvst4,
    `1` = 5,
    `2` = 4,
    `3` = 3,
    `4` = 2,
    `5` = 1
  ))
```

Score the Love of Statistics Scale instrument

Run the following code block to calculate a final score for the Love of Statistics Scale for each participant.

- What level of measurement is the new LOSS_total
  - Answer:

```
dataset <- dataset %>% mutate(
  lvst1_num = recode(lvst1,
    "Not at all"  = 1,
    "a little"    = 2,
    "moderately"  = 3,
    "mostly"      = 4,
    "completely"  = 5
  ),
  lvst2_num = recode(lvst2,
    "Not at all"  = 1,
    "a little"    = 2,
    "moderately"  = 3,
    "mostly"      = 4,
    "completely"  = 5
  ),
  lvst3_num = recode(lvst3,
    "Not at all"  = 1,
    "a little"    = 2,
    "moderately"  = 3,
    "mostly"      = 4,
    "completely"  = 5
  ),
  lvst4reverse_num = recode(lvst4reverse,
```

```
    "Not at all"  = 1,
    "a little"    = 2,
    "moderately"  = 3,
    "mostly"      = 4,
    "completely"  = 5
  ),
  LOSS_total = rowSums(cbind(lvst1_num, lvst2_num, lvst3_num, lvst4reverse_num), na.rm = TRUE)

)
```

```
## Warning: There was 1 warning in `mutate()`.
## i In argument: `lvst4reverse_num = recode(...)`.
## Caused by warning in `recode.numeric()`:
## ! NAs introduced by coercion
```

Score the Fear of Statistics Scale instrument

Enter code in the following code block to calcuate a final score for the Fear of Statistics Scale for each participant. Call the new variable FOSS_total.

```
# Enter code here below the comment.
dataset <- dataset %>% mutate(FOSS_total = as.numeric(frst1) + as.numeric(frst2) + as.numeric(frst3))
```

Examine the new LOSS_total and FOSS_total variables

Note any issues with the data in the new variables. Place code you use to examine the variables in the code block below. (Or just describe what you did.)

```
# look at the structure and quick summaries
summary(dataset$LOSS_total)
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##   3.000   7.000   9.000   8.896  10.250  14.000
```

```
summary(dataset$FOSS_total)
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##   3.000   7.000   8.000   8.208  10.000  14.000
```

```
# check for missing values
sum(is.na(dataset$LOSS_total))
```

```
## [1] 0
```

```
sum(is.na(dataset$FOSS_total))
```

```
## [1] 0
```

```r
# check the actual values (are they in a reasonable range?)
range(dataset$LOSS_total, na.rm = TRUE)
```

```
## [1]  3 14
```

```r
range(dataset$FOSS_total, na.rm = TRUE)
```

```
## [1]  3 14
```

```r
# show what lvst values actually look like
head(dataset$lvst1, 15)
```

```
##  [1] moderately moderately a little   a little   a little   moderately
##  [7] moderately moderately moderately a little   moderately moderately
## [13] mostly     a little   a little
## Levels: Not at all a little moderately mostly completely
```

```r
head(dataset$lvst4reverse, 15)
```

```
##  [1] NA NA NA NA NA NA NA NA NA NA NA NA NA NA NA
```

```r
# show unique values (or at least the most common ones)
table(dataset$lvst1, useNA = "ifany")
```

```
##
## Not at all   a little moderately     mostly completely
##          5         29         42         15          5
```

```r
table(dataset$lvst4reverse, useNA = "ifany")
```

```
##
## <NA>
##   96
```

```r
summary(dataset$LOSS_mean)
```

```
## Length  Class   Mode
##      0   NULL   NULL
```

```r
summary(dataset$FOSS_mean)
```

```
## Length  Class   Mode
##      0   NULL   NULL
```

Final scores using averages

Not all instruments are scored using sums. If all items are measured using the same scale a mean can be calculated. One advantage of using means is the final score is in the same scale as the original items so interpreting can be a bit more straightforward, It's also possible to calculate the mean when there are missing data without changing the scale of the final score. Run the following code block to score the instruments using means instead of sums.

```
dataset <- dataset %>% mutate(LOSS_mean = (as.numeric(lvst1) + as.numeric(lvst2) + as.numeric(lvst3) + a
dataset <- dataset %>% mutate(FOSS_mean = (as.numeric(frst1) + as.numeric(frst2) + as.numeric(frst3))/3)
```

z-scores

Z-scores are useful for a number of reasons. Since z-scores are unit free (the units are standard deviations) they can be used to compare scores between instruments or tests which have different units. (Like our LOSS scale which is in love units and our FOSS scale which is in fear units.) Also, z-scores let us see how many standard deviations a score is away from the mean. It is an easy way to see which scores are potenential outliers. Z-scores above 2 are more than 2 standard deviations away from the mean and are potential univariate outliers.Run the following code block to calculate z-scores for the LOSS and FOSS variables. Examine the new variables.

- How do the z-scores compare between the sum scores and the mean scores?
  - Answer: They are the same or similar b/c z scores standardize the values.
- Are there any z-scores greater than 2 in the LOSS and FOSS variables? How many?
  - LOSS z-score > 2:0
  - FOSS z-score > 2:0
- What does it mean if a z-score is positive or negative?
  - positive: the score is above the mean
  - negative: the scre is below the mean

```
dataset <- dataset %>% mutate(LOSS_total_z = scale(LOSS_total))
dataset <- dataset %>% mutate(LOSS_mean_z = scale(LOSS_mean))

dataset <- dataset %>% mutate(FOSS_total_z = scale(FOSS_total))
dataset <- dataset %>% mutate(FOSS_mean_z = scale(FOSS_mean))
```

Save your data for next week

```
rio::export(dataset,"Week4.rds")
```

Save your output.

Save this assignment as a markdown file.

Submit your assignment.

Submit the output you just saved for your assignment.